Foundations of AI Capstone Project

Wednesday, May 13, 2026 8:43 AM

Designing a Knowledge-Based Expert System

Topic: Coffee Drink Recommendation Expert System

1. Introduction and Domain Definition

Introduction

A Knowledge-Based Expert System is an artificial intelligence system that uses logical rules and stored knowledge to solve problems that normally require human expertise. These systems imitate the reasoning process of experts and provide recommendations or decisions based on facts and rules.

This project designs a theoretical expert system for recommending coffee drinks to users according to their personal preferences, lifestyle, and taste choices. The system does not require coding implementation. Instead, it demonstrates the principles of symbolic AI, knowledge representation, rule-based reasoning, and inference strategies.

Domain Definition

The chosen domain for this expert system is:

Coffee Drink Recommendation System

The system acts like a coffee expert that suggests the most suitable coffee drink for a user.

Problem Statement

Many people are unsure which coffee drink best matches their preferences. Different drinks vary in sweetness, caffeine level, milk content, flavor strength, and temperature.

The goal of this expert system is to:

- Analyze user preferences.
- Apply expert knowledge using logical rules.
- Recommend the most suitable coffee drink.
- Avoid unsuitable recommendations.

The final output of the system will be a recommendation such as:

- Recommend(Espresso)
- Recommend(Cappuccino)
- Recommend(Latte)
- Recommend(ColdCoffee)

2. Knowledge Representation

Key Concepts in the Domain

The main entities and concepts used in the system are:

User Properties

- LikesStrongCoffee(User)
- LikesSweetTaste(User)
- PrefersColdDrink(User)
- WantsHighCaffeine(User)
- PrefersMilk(User)
- IsBeginner(User)
- WantsLowSugar(User)

Coffee Drink Properties

- IsStrong(Espresso)
- HasMilk(Latte)
- HasFoam(Cappuccino)
- IsCold(ColdCoffee)
- IsSweet(Mocha)
- HasHighCaffeine(Espresso)
- IsLowSugar(Americano)

Recommendation Relations

- IsSuitable(Drink, User)
- NotSuitable(Drink, User)
- Recommend(Drink)

Formal Knowledge Representation

The system stores knowledge using predicate logic.

Examples:

- LikesStrongCoffee(John)
- PrefersMilk(John)
- HasMilk(Latte)
- IsStrong(Espresso)
- Recommend(Latte)

This structure allows the system to reason logically using IF-THEN rules.

3. The Knowledge Base: Rules

The expert knowledge is represented using IF-THEN logical rules.

Rule Set

RULE 1

IF LikesStrongCoffee(User) AND WantsHighCaffeine(User) THEN IsSuitable(Espresso, User)

RULE 2

IF PrefersMilk(User) AND LikesSweetTaste(User) THEN IsSuitable(Latte, User)

RULE 3

IF PrefersMilk(User) AND LikesStrongCoffee(User) THEN IsSuitable(Cappuccino, User)

RULE 4

IF PrefersColdDrink(User) THEN IsSuitable(ColdCoffee, User)

RULE 5

IF WantsLowSugar(User) THEN IsSuitable(Americano, User)

RULE 6

IF IsBeginner(User) AND PrefersMilk(User) THEN IsSuitable(Latte, User)

RULE 7

IF LikesSweetTaste(User) AND WantsHighCaffeine(User) THEN IsSuitable(Mocha, User)

RULE 8

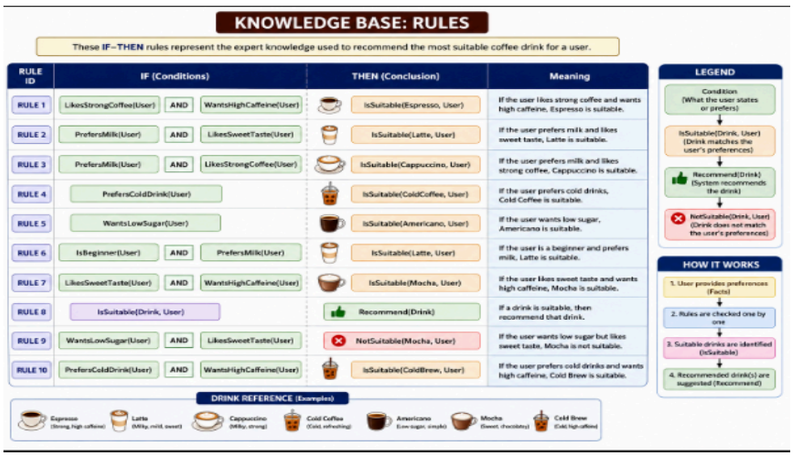
IF IsSuitable(Drink, User) THEN Recommend(Drink)

RULE 9

IF WantsLowSugar(User) AND LikesSweetTaste(User) THEN NotSuitable(Mocha, User)

RULE 10

IF PrefersColdDrink(User) AND WantsHighCaffeine(User) THEN IsSuitable(ColdBrew, User)



4. Inference Strategy

Chosen Strategy: Forward Chaining

This expert system uses the **Forward Chaining** inference strategy.

Forward chaining is a data-driven reasoning process. The system starts with known facts provided by the user and continuously applies rules to derive new conclusions until a recommendation is reached.

Why Forward Chaining Was Chosen

Forward chaining is suitable for this project because:

1. The system begins with user inputs such as taste preferences and drink choices.
2. The system checks all possible rules using these facts.
3. New conclusions are generated step by step.
4. The process naturally leads to one or more coffee recommendations.

This approach is efficient for recommendation systems because users provide facts first, and the system determines the best result from available rules.

5. Sample Consultation: Step-by-Step Trace

Scenario

A user named John visits the coffee recommendation system.

Initial Facts Provided by the User

The user gives the following information:

- LikesStrongCoffee(John)
- PrefersMilk(John)
- WantsHighCaffeine(John)
- LikesSweetTaste(John)

Step-by-Step Inference Process

Step 1: Known Facts

The system starts with:

- LikesStrongCoffee(John)
- PrefersMilk(John)
- WantsHighCaffeine(John)
- LikesSweetTaste(John)

Step 2: Check RULE 1

RULE 1 states:

IF LikesStrongCoffee(User) AND WantsHighCaffeine(User) THEN IsSuitable(Espresso, User)

The conditions are satisfied because:

- LikesStrongCoffee(John) = TRUE
- WantsHighCaffeine(John) = TRUE

Therefore:

New Fact Derived:

- IsSuitable(Espresso, John)

Step 3: Check RULE 2

RULE 2 states:

IF PrefersMilk(User) AND LikesSweetTaste(User) THEN IsSuitable(Latte, User)

The conditions are satisfied because:

- PrefersMilk(John) = TRUE
- LikesSweetTaste(John) = TRUE

Therefore:

New Fact Derived:

- IsSuitable(Latte, John)

Step 4: Check RULE 3

RULE 3 states:

IF PrefersMilk(User) AND LikesStrongCoffee(User) THEN IsSuitable(Cappuccino, User)

The conditions are satisfied because:

- PrefersMilk(John) = TRUE
- LikesStrongCoffee(John) = TRUE

Therefore:

New Fact Derived:

- IsSuitable(Cappuccino, John)

Step 5: Check RULE 7

RULE 7 states:

IF LikesSweetTaste(User) AND WantsHighCaffeine(User) THEN IsSuitable(Mocha, User)

The conditions are satisfied because:

- LikesSweetTaste(John) = TRUE
- WantsHighCaffeine(John) = TRUE

Therefore:

New Fact Derived:

- IsSuitable(Mocha, John)

Step 6: Apply RULE 8

RULE 8 states:

IF IsSuitable(Drink, User) THEN Recommend(Drink)

The system now has:

- IsSuitable(Espresso, John)
- IsSuitable(Latte, John)
- IsSuitable(Cappuccino, John)
- IsSuitable(Mocha, John)

Therefore the following recommendations are generated:

- Recommend(Espresso)
- Recommend(Latte)
- Recommend(Cappuccino)
- Recommend(Mocha)

Final Conclusion

The expert system concludes that the best coffee choices for John are:

1. Espresso
2. Latte
3. Cappuccino
4. Mocha

Among these options, Cappuccino and Mocha are considered the most balanced choices because they satisfy multiple preferences simultaneously.

6. Conclusion and Limitations

Conclusion

This project successfully designed a theoretical knowledge-based expert system for recommending coffee drinks. The system uses symbolic AI concepts such as:

- Knowledge representation
- Logical IF-THEN rules
- Predicate logic
- Forward chaining inference

The expert system demonstrates how AI systems can mimic human expert reasoning to provide useful recommendations.

Limitations

Although the system works well for simple recommendations, it has several limitations:

1. Cannot Handle Uncertainty

The system only works with exact facts. It cannot handle uncertain or incomplete information.

2. No Learning Ability

The system does not learn from experience or user feedback.

3. Limited Knowledge Base

The recommendations are restricted to predefined rules and drinks.

4. Conflict Between Rules

Sometimes multiple rules may recommend different drinks at the same time.

5. No Natural Language Understanding

The system requires structured inputs and cannot fully understand human language.

Final Summary

The Coffee Drink Recommendation Expert System demonstrates the core principles of knowledge-based AI systems. By combining logical rules, formal knowledge representation, and forward chaining reasoning, the system can successfully analyze user preferences and generate intelligent coffee recommendations.

This project provides a strong understanding of expert systems, symbolic reasoning, and inference mechanisms without requiring programming implementation.

Thank You

Second Project

Manual Graph Traversal Analysis (DFS & BFS)

I. Graph Definition and Setup

1. Graph Diagram

The graph used for this project contains 9 nodes:

A, B, C, D, E, F, G, H, I

The graph is undirected and contains cycles.

Graph Connections

- A connected to B and C
- B connected to D and E
- C connected to F and G
- D connected to H
- E connected to H
- F connected to I
- G connected to I

This structure creates cycles such as:

- D → H → E → B
- F → I → G → C

2. Adjacency List

A: [B, C]

B: [A, D, E]

C: [A, F, G]

D: [B, H]

E: [B, H]

F: [C, I]

G: [C, I]

H: [D, E]

I: [F, G]

3. Search Parameters

Starting Node

The starting node for both DFS and BFS is:

- Start Node = A

Tie-Breaking Rule

Whenever multiple neighboring nodes are available, the algorithm always explores nodes in:

- Alphabetical Order

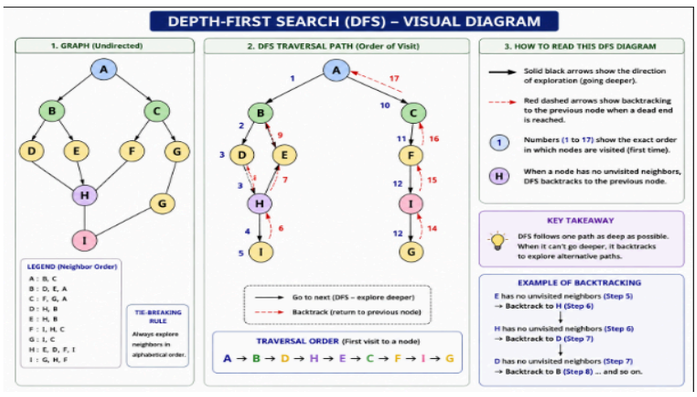
This ensures a predictable and consistent traversal sequence.

II. Depth-First Search (DFS) Analysis

1. DFS Trace Table

Step #	Current Node	Visited Set	Action Taken / Next Node	Backtrack?
1	A	{A}	Visit A. Move to B	No
2	B	{A, B}	Visit B. Move to D	No
3	D	{A, B, D}	Visit D. Move to H	No
4	H	{A, B, D, H}	Visit H. Move to E	No
5	E	{A, B, D, H, E}	E has no unvisited neighbors	Yes
6	H	{A, B, D, H, E}	Return to H. No unvisited neighbors	Yes
7	D	{A, B, D, H, E}	Return to D. No unvisited neighbors	Yes
8	B	{A, B, D, H, E}	Return to B. No unvisited neighbors	Yes
9	A	{A, B, D, H, E}	Move to C	No
10	C	{A, B, C, D, E, H}	Visit C. Move to F	No
11	F	{A, B, C, D, E, F, H}	Visit F. Move to I	No
12	I	{A, B, C, D, E, F, H, I}	Visit I. Move to G	No
13	G	{A, B, C, D, E, F, G, H, I}	G has no unvisited neighbors	Yes
14	I	{A, B, C, D, E, F, G, H, I}	Return to I. No unvisited neighbors	Yes
15	F	{A, B, C, D, E, F, G, H, I}	Return to F. No unvisited neighbors	Yes

16	C	{A, B, C, D, E, F, G, H, I}	Return to C. No unvisited neighbors	Yes
17	A	{A, B, C, D, E, F, G, H, I}	DFS Complete	Yes



2. Final DFS Traversal Sequence

A → B → D → H → E → C → F → I → G

3. DFS Analysis

The DFS traversal demonstrates the LIFO (Last-In, First-Out) behavior because the algorithm explores one branch completely before moving to another branch.

Example of Backtracking 1

At node E:

- E had no unvisited neighbors.
- The algorithm backtracked to H.
- This occurred because the branch was fully explored.

Example of Backtracking 2

At node G:

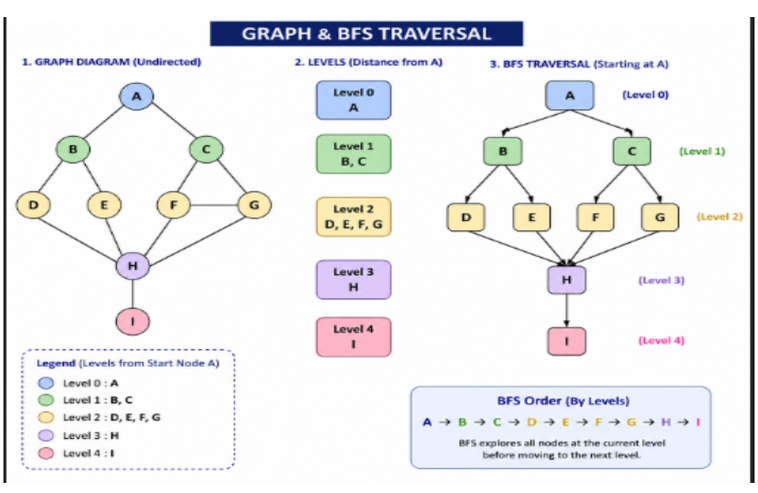
- G had no unvisited neighbors.
- The algorithm backtracked to I and then to F.
- DFS continued returning backward until another unexplored branch was found.

These examples clearly show how DFS moves deeply into the graph before returning to previous nodes.

III. Breadth-First Search (BFS) Analysis

1. BFS Trace Table

Step #	Current Node (Dequeued)	Queue Contents (Before Dequeue)	Visited Set	Action Taken / Nodes Enqueued
1	A	[A]	{A}	Enqueue B, C
2	B	[B, C]	{A, B, C}	Enqueue D, E
3	C	[C, D, E]	{A, B, C, D, E}	Enqueue F, G
4	D	[D, E, F, G]	{A, B, C, D, E, F, G}	Enqueue H
5	E	[E, F, G, H]	{A, B, C, D, E, F, G, H}	No new nodes
6	F	[F, G, H]	{A, B, C, D, E, F, G, H}	Enqueue I
7	G	[G, H, I]	{A, B, C, D, E, F, G, H, I}	No new nodes
8	H	[H, I]	{A, B, C, D, E, F, G, H, I}	No new nodes
9	I	[I]	{A, B, C, D, E, F, G, H, I}	BFS Complete



2. Final BFS Traversal Sequence

A → B → C → D → E → F → G → H → I

3. BFS Analysis

The BFS traversal demonstrates FIFO (First-In, First-Out) behavior because nodes are explored in the exact order they enter the queue.

The queue ensures that all nodes at depth k are explored before moving to nodes at depth k+1.

Example

- Starting from A, the algorithm first visits all immediate neighbors:
  - B and C
- Only after completing depth 1 does BFS move to depth 2:
  - D, E, F, and G
- Finally, BFS explores deeper nodes:
  - H and I

This level-by-level exploration guarantees systematic traversal across the graph.

IV. Conclusion and Comparison

1. Path Optimality

Breadth-First Search (BFS) guarantees the shortest path in terms of the number of edges.

This happens because BFS explores nodes level-by-level.

If the target node is far from the start node, BFS will always find the shortest possible path first.

Depth-First Search (DFS) does not guarantee the shortest path because it explores one branch deeply before checking alternative paths.

Therefore:

- BFS = Shortest path guaranteed
- DFS = Shortest path not guaranteed

2. Memory Complexity

DFS is generally more memory-efficient for graphs that are very deep with a low branching factor.

Reason

DFS only stores:

- The current traversal path
- A small number of neighboring nodes

BFS must store all nodes at the current level in the queue.

In wide graphs, the BFS queue can become very large.

Therefore:

- DFS uses less memory in deep graphs.
- BFS requires more memory because it stores many frontier nodes simultaneously.

Final Summary

This project demonstrated the practical application of DFS and BFS graph traversal algorithms.

Key observations:

- DFS explores deeply and uses backtracking.
- BFS explores level-by-level using a queue.
- BFS guarantees the shortest path.
- DFS is usually more memory-efficient.

The manual trace tables clearly illustrate how both algorithms process graphs differently while solving search problems in Artificial Intelligence.

Thank You
b25bs1446